

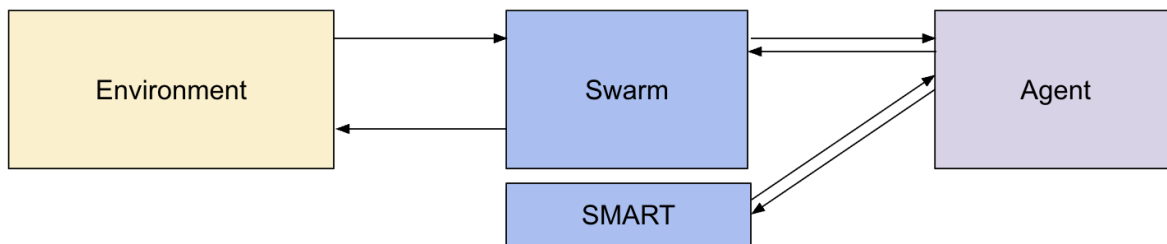
Introduction:

This project implements a custom capture-the-flag environment for multi-domain (ground + air) swarms. It furthermore includes a shared situational awareness layer per swarm, enabling individual agents within the swarm, and the operator controlling the swarm, to have a global understanding of the environment. This project also supports multi-agent reinforcement learning, with baseline policies and reward functions implemented.

My overall goal with this project was to explore the problem of coordinating multi-agent, heterogeneous swarms from the ground up. I initially attempted to find already-implemented multi-agent frameworks for this project, but there were none that could be adapted to my intended idea without significant effort. I opted to implement this environment from scratch to maintain full flexibility over the system design and tailor it to my idea. I also learned a lot :)

The following is a high-level overview of the core classes of the environment.

Environment Loop



The Environment serves as the ground-truth for the simulation. At each step of the rollout, we pass its underlying array to the Swarm's, which calculate appropriate observation slices for each Agent (based on position and observation radius). While it would be purer to have the Environment calculate observation slices and leave the Swarm and Agents as just observers, I opted to keep perception logic at the swarm level to allow for modularity and flexibility in experimentation.

The Swarm also receives “feedback” from the environment, namely relating to events and the rewards associated with them. The Swarm can then update the SMART with such events, as well as the rewards of each agent. The observation slices and relevant SMART information are then inputted to each Agent. Agents report entity observations (updating the SMART) and use their policy to output a Move action and/or an Engage action. These actions are compiled by the swarm, and are sent to the environment, which executes them and updates the underlying ground truth array. This process then starts again.

Environment

The Environment class is responsible for tracking the ground-truth state and executing actions provided to it by the swarms. Fundamentally, it is based on a four-channel numpy array. These are as follows:

- 0: Terrain/heightmap
- 1: Agents: (1, 2), (3, 4) --> (Ground1, Air1), (Ground2, Air2)
- 2: Agent health
- 3: Flag dispositions, floats in [-1, 1]

The heightmap is generated using Perlin noise. At instantiation, the Environment generates N heightmaps (N is configurable), each with different seeds. At the start of each rollout on `reset()`, the Environment randomly chooses a generated heightmap. For agent and flag observations, we flip them accordingly on the Agent level to ensure symmetric observations between opposing swarms. This ensures that (1,2) always indicates friendly agents and 1 always means that a flag's disposition is friendly.

The environment is built around having two opposing swarms. It initializes their positions on the starting and ending fifths of the map, respecting terrain constraints (e.g. spawning ground agents in relatively flat regions). Furthermore, we ensure that agents are sufficiently spaced out. Flags are initialized in the middle 40% of the map, with half being in flat regions and the other half in high/low elevation regions. The goal of this is to ensure that agents are capable of capturing flags in varying settings, and encourage the use of the heterogeneous nature of the swarms.

Another crucial role of the environment is executing actions. Possible actions include Move and Engage (specified below). These actions are parsed by the environment and the ground-truth array (e.g. agent positions, health, flag dispositions) are updated accordingly. The associated rewards with the events caused by these actions (and the events themselves) are recorded as feedback for the swarms. For example, if an agent moves into the capture radius of a flag, that flag's disposition and the agent's position must be updated.

Now suppose this moved agent was engaged by an opposing agent. The target coordinates of this Engage action would be outdated, as the target agent has moved. For simplicity, I assume that engagements are adaptive to such changes (e.g. imagine a sophisticated, trajectory-adjusting missile). I record the past frame's agent positions, and use the target coordinates to find the intended target agent's id, which I use to get the target agent's current position.

I initially implemented damage kernels which affected greater areas but had the potential for friendly fire. In initial RL training runs, I found that agents were hesitant to engage due to this risk. With the currently implemented precision engagements, I am seeing that agents are much more eager to engage due to there being virtually no risk but a moderately high reward.

However this has resulted in them not completing what should be the primary objective, capturing flags. Refining the rewards to address this is an ongoing process.

Swarm Setup

The Swarm keeps track of the overall situational awareness of the swarm via its SMART (more below), and is responsible for calling Agents and collecting their outputted actions, which are then sent to be executed by the Environment. Another crucial role of the Swarm is receiving feedback from the environment. The Swarm adds events to the SMART and updates the rewards of each agent for the current step. It also keeps track of active and eliminated agents, ensuring that only active agents are called to make actions.

Agent Setup

The agent is responsible for processing the observations that it has been provided, and outputting its actions for that timestep. The two possible action types are:

- Move: (NOOP + 8 directions, magnitude determined by max speed of the agent)
- Engage (target x, target y, damage amount determined by engager type)

These actions are then collected by the swarm and are subsequently sent to the environment to be executed.

With respect to processing observations, the agent fundamentally reports entity observations to the SMART situational awareness layer (detailed below).

Agent types are either ground or air. Ground agents have higher health, damage, and flag capture speed but lower speed and observation radii. Air agents have lower health, flag capture speed, and damage, but higher speed and observation radii. The goal with this heterogeneity is to encourage the discovery of unique strategies that take advantage of the pros of each type while balancing the cons.

SMART (Situational Awareness Layer)

Swarm-based Multi-agent Awareness and Real-time Tracking (SMART) is a swarm-level situational awareness layer that keeps track of foreign entities and events happening in the environment. It provides information based on recency, priority, and distance to agents as observations and is updated by both the swarm (for events) and agents (for entity observation).

Event types include:

- Flag captures by friendly agents
- Enemy agent discoveries
- Friendly agents engaging enemy agents
- Friendly agents taking damage
- Friendly agents being eliminated

Along with this, the possible entity dispositions are friendly and enemy. In future iterations of this environment I may add more dispositions such as unknown or suspicious. Each entity and event has a timestamp associated with when it was last seen/created. At the end of each timestep, SMART removes stale entities/events which are beyond the set time-to-live threshold to ensure freshness of information.

When Agents report entity observations to the SMART, it uses the Hungarian matching algorithm to match currently tracked foreign entities with these observations, and creates new entity tracks if necessary. Only entities matching the observation type and within a set match radius (the maximum speed of the observation type) are eligible to be matched. This can definitely be tuned to have constraints based on the staleness of the entity. Having the match radius be the maximum speed of an entity may also be too loose of a constraint. In future iterations of this environment, I will investigate including velocity as an observable attribute, which would possibly allow for more robust tracking.

Deterministic Strategies

These were simple strategies I implemented to demonstrate and test my environment – perhaps “demo strategies” would be a better way to describe them. Demo videos can be found in the /demos folder in my repository.

The Swarm and Agent classes I described above were geared toward the use of a neural network acting at the level of each agent. However, for simplicity, I implemented these deterministic strategies at the swarm level, so I created the **DeterministicSwarm** and **DeterministicAgent** subclasses, and removed the use of a policy at the agent level.

Instead, the role of the DeterministicAgent is to process its local observations and report them to the swarm-level controller, which then creates Move or Engage actions for that agent. For example, the DeterministicAgent might report the presence of foreign entities and their coordinates, and the swarm-level controller would then create an Engage action targeting one of these entities. This list of actions is then readily passed into the Environment for execution.

Implemented strategies are as follows:

- Distributed: Equally-sized contingents of agents are each assigned to a flag at the start of a rollout. They make their way to their target flags and stay there, engaging foreign entities if detected.
- Liquid: The entire swarm moves as one to the closest flag, then to the closest flag from there, and so on. At each flag, a contingent of agents is left behind to defend the flag. Agents engage foreign entities if they are detected.
- Forward: All agents move forward (no engage actions).
- Passive: All agents stay stationary.

In the future, an LLM-based approach could be used to coordinate the swarm, following the DeterministicSwarm model where agents report their local observations to the level of the swarm. This would heavily depend on formatting SMART information properly for the prompt and the spatio-temporal reasoning capability of the model.

More to come, RL baseline training is ongoing. Reward function tuning is in-progress, and I am experimenting with fictitious self-play and parallel learning. The goal is to have a swarm consistently capture all flags against deterministic strategies.